A

Mini Project Report

on

Securio

Submitted in partial fulfillment of the requirements for the degree

Second Year Engineering – Computer Science Engineering (Data Science)

by

Pratiksha Jadhav	**24107107**
Shravani Bhilare	**24107112**
Srushti Balsaraf	**24107096**
Sakshi Kale	**24107071**

Under the guidance of

Prof .Veena Gawde

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (DATA SCIENCE)

A.P. SHAH INSTITUTE OF TECHNOLOGY

G.B. Road, Kasarvadavali, Thane (W)-400615

UNIVERSITY OF MUMBAI

Academic year: 2025-26

CERTIFICATE

This to certify that the Mini Project report on Securio has been submitted by Pratiksha Jadhav(24107107),Shravani Bhilare (24107112), Srushti Balsaraf(24107096) and Sakshi Kale(24107071) who are bonafide students of A. P. Shah Institute of Technology, Thane as a partial fulfillment of the requirement for the degree in **Computer Science Engineering (Data Science)**, during the academic year **2025-2026** in the satisfactory manner as per the curriculum laid down by University of Mumbai.

Prof.Veena Gawde
Guide

Dr. Pravin Adivarekar
HOD, CSE(Data Science)

Dr. Uttam D. Kolekar
Principal

External Examiner:

1.

Internal Examiner:

1.

Place: A. P. Shah Institute of Technology, Thane

Date:

ACKNOWLEDGEMENT

This project would not have come to fruition without the invaluable help of our guide Prof.Veena Gawde Expressing gratitude towards our HoD, **Dr. Pravin Adivarekar**, and the Department of Computer Science Engineering (Data Science) for providing us with the opportunity as well as the support required to pursue this project. We would also like to thank our project coordinator **Prof. Dipali Ghat and Prof Hardiki Patil** who gave us her valuable suggestions and ideas when we were in need of them. We would also like to thank our peers for their helpful suggestions.

TABLE OF CONTENTS

1. Introduction… 1

1.1. Purpose 1

1.2. Problem Statement 2

1.3. Objectives 3

1.4. Scope 3

2. Proposed System 4

2.1. Features and Functionality 4

3. Project Outcomes 6

4. Software Requirements 7

5. Project Design 8

6. Project Scheduling 10

7. Results 11

8. Conclusion 13

References

Chapter 1: Introduction

In today's digital era, ensuring security and efficient management in residential societies has become increasingly important. However, many apartment complexes still rely on manual methods such as maintaining visitor registers, calling residents for approvals, and handling complaints and maintenance records offline. These traditional systems are not only time-consuming but also lack transparency, real-time communication, and proper data management, leading to security risks and operational inefficiencies.

The project **Securio – Apartment Security & Management App** aims to provide a centralized and digital solution to overcome these challenges. The application focuses on improving apartment security through a structured visitor management system while enabling seamless communication between guards, residents, and administrators. By digitizing these processes, the system reduces manual effort and enhances overall efficiency.

The app introduces a role-based approach where Admin, Guard, and Owner have specific functionalities. Guards can register visitors and notify owners instantly, owners can approve or reject entries in real-time, and admins can manage complaints, maintenance, and user data effectively. This ensures smooth coordination and faster decision-making within the society.

The major contribution of this project is the integration of key apartment management features into a single platform, including real-time visitor approval, complaint handling with status tracking, maintenance monitoring, and house helper management. This system improves security, ensures transparency, and provides a modern and user-friendly solution for apartment management.

1.1 Purpose:

The purpose of this document is to present a detailed description of the **Securio – Apartment Security & Management App**, including its design, features, functionality, and implementation. It serves as a comprehensive guide to understand how the system works, the problems it solves, and the technologies used in its development.

This document is intended for multiple stakeholders, including:

- **Project Evaluators and Faculty:** To assess the design, implementation, and outcomes of the project
- **Developers:** To understand the system architecture, modules, and workflow for further development or maintenance
- **Apartment Administrators:** To understand how the application can be used for efficient society management
- **Residents (Owners):** To gain insight into how the system improves security, communication, and convenience
- **Security Personnel (Guards):** To understand the usage of the visitor management system

Overall, this document acts as a reference for understanding, evaluating, and potentially enhancing the application.

1.2 Problem Statement:

In many residential apartment societies, daily operations such as visitor management, complaint handling, maintenance tracking, and communication are still managed using manual and outdated systems. These methods involve physical registers, phone calls, and paper-based records, which are inefficient, time-consuming, and prone to human errors.

The lack of a proper digital system leads to several issues, including poor visitor verification, delays in communication between guards and residents, lack of transparency in complaint resolution, and difficulty in tracking maintenance records. Additionally, there is no structured mechanism to manage and monitor house helpers, which raises security concerns. Due to these limitations, there is a need for a centralized and automated solution that ensures secure data handling, real-time communication, and efficient management of apartment activities.

1.3 Objectives:

1. To prevent unauthorized access in residential buildings.
2. To enable real-time resident approval for visitor entry.
3. To digitize and automate the visitor entry process.
4. To maintain accurate visitor entry logs for future reference

1.4 Scope

The scope of this project is limited to developing a mobile-based solution for apartment societies. The application covers:

1. Can be applied in residential societies, apartments, and gated communities for digital visitor management.

2. Can be useful to security staff for registering and tracking visitor entries efficiently.

3. Can assist residents in approving or rejecting visitors through a mobile application.

4. Can be extended in future to include features like emergency alerts and advanced security monitoring.

Chapter 2: Proposed System

The proposed system is a smart apartment management application that automates all major operations of a residential society. It replaces manual processes with a digital system that ensures better security, faster communication, and efficient data management.The system uses Firebase for authentication and real-time database management, while Cloudinary is used for efficient image handling.

2.1 Features and Functionality

- **Visitor Management:**

  Guards can add visitor details, and owners can approve or reject requests instantly.

- **Real-Time Communication:**

  Data is updated instantly using Firebase, ensuring no delay in approvals.

- **Complaint Management:**

  Owners can raise complaints, and admins can update their status

- **Maintenance Tracking:**

  Admin can assign maintenance charges, and owners can view dues and payment status.

- **Helper Management:**

  Owners can add house helpers and control their entry permissions.

- **Image Handling:**

  Visitor images can be managed using Cloudinary for better performance and storage.

Chapter 3: Project Outcomes

The implementation of the **Securio – Apartment Security & Management App** results in significant improvements in the overall functioning of residential societies. By replacing manual processes with a digital system, the application enhances efficiency, security, and user experience.

Key Outcomes:

- **Improved Apartment Security:**

  Digital visitor tracking ensures proper recording, verification, and monitoring of all entries and exits, reducing unauthorized access.

- **Faster Communication Between Users:**

  Real-time updates enable instant communication between guards, owners, and administrators, eliminating delays caused by phone calls.

- **Reduced Dependency on Manual Processes:**

  Automation of tasks such as visitor entry, complaint handling, and maintenance tracking minimizes paperwork and human effort.

- **Increased Transparency:**

  Complaints and maintenance records are digitally stored and updated, allowing users to track status easily and ensuring accountability.

- **Better Management of House Helpers:**

  A structured system for registering and monitoring helpers ensures controlled access and improved security.

Chapter 4: Software Requirements

The software components used in the development of the application include:

- Frontend: Android (Java/Kotlin, XML for UI design)
- Backend: Firebase Firestore (real-time database)
- Authentication: Firebase Authentication
- Notifications: Firebase Cloud Messaging (FCM)
- Image Storage: Cloudinary (for storing and optimizing images)
- IDE: Android Studio
- Operating System: Windows / Linux

Chapter 5: Project Design

The system follows a **client-server architecture**, ensuring scalability and real-time performance.

Architecture Components:

- **Client:** Android Application
- **Authentication Layer:** Firebase Authentication
- **Database Layer:** Firestore
- **Notification Layer:** FCM
- **Image Storage Layer:** Cloudinary

Modules:

- Visitor Management Module
- Complaint Management Module
- Maintenance Module
- Helper Management Module

System Flow:

1. User logs into the application
2. Role-based dashboard is displayed

3. User performs actions (add visitor, complaint, etc.)

4. Data is stored in Firestore

5. Real-time updates are sent to other users

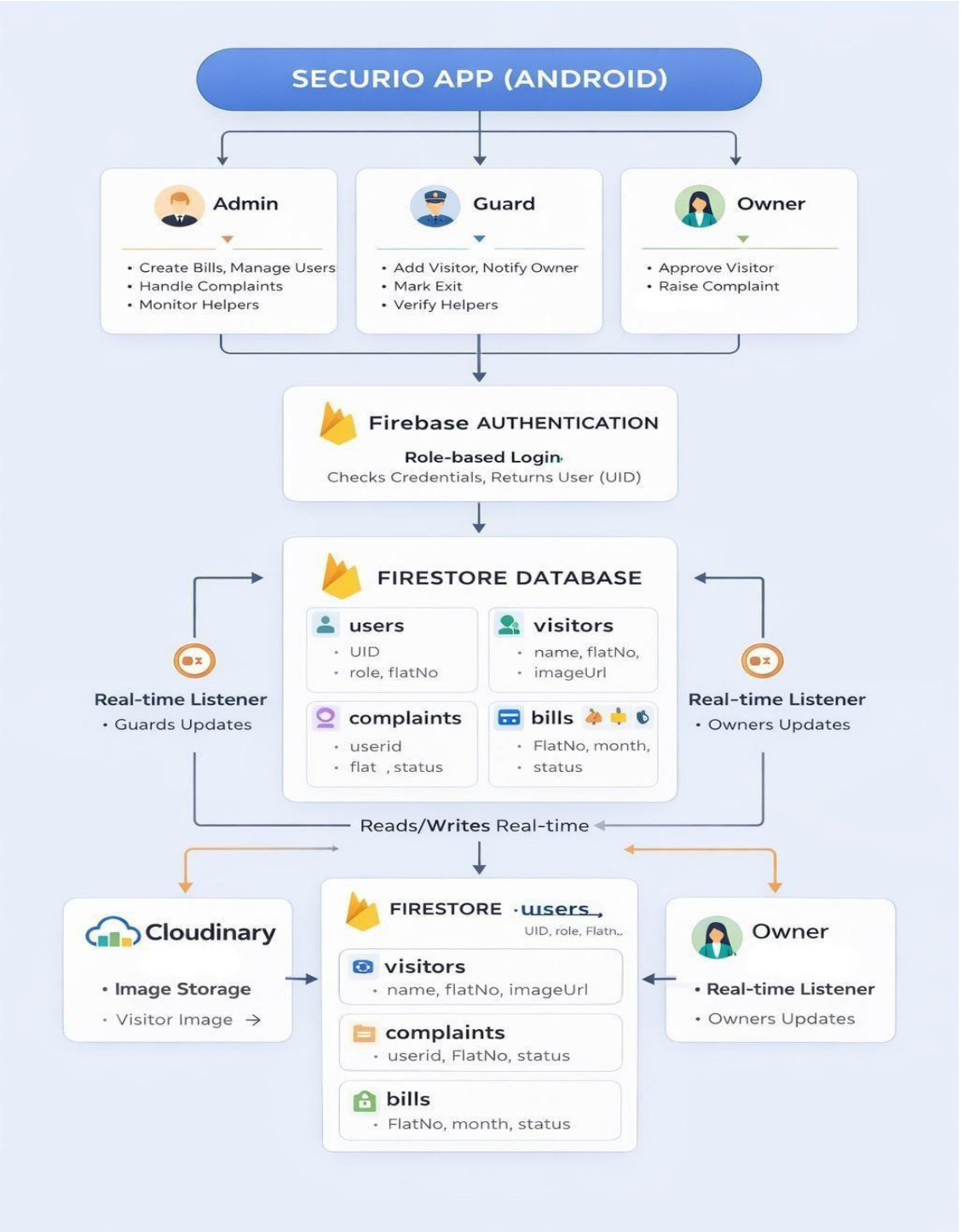


Figure 5.1: Systemflow

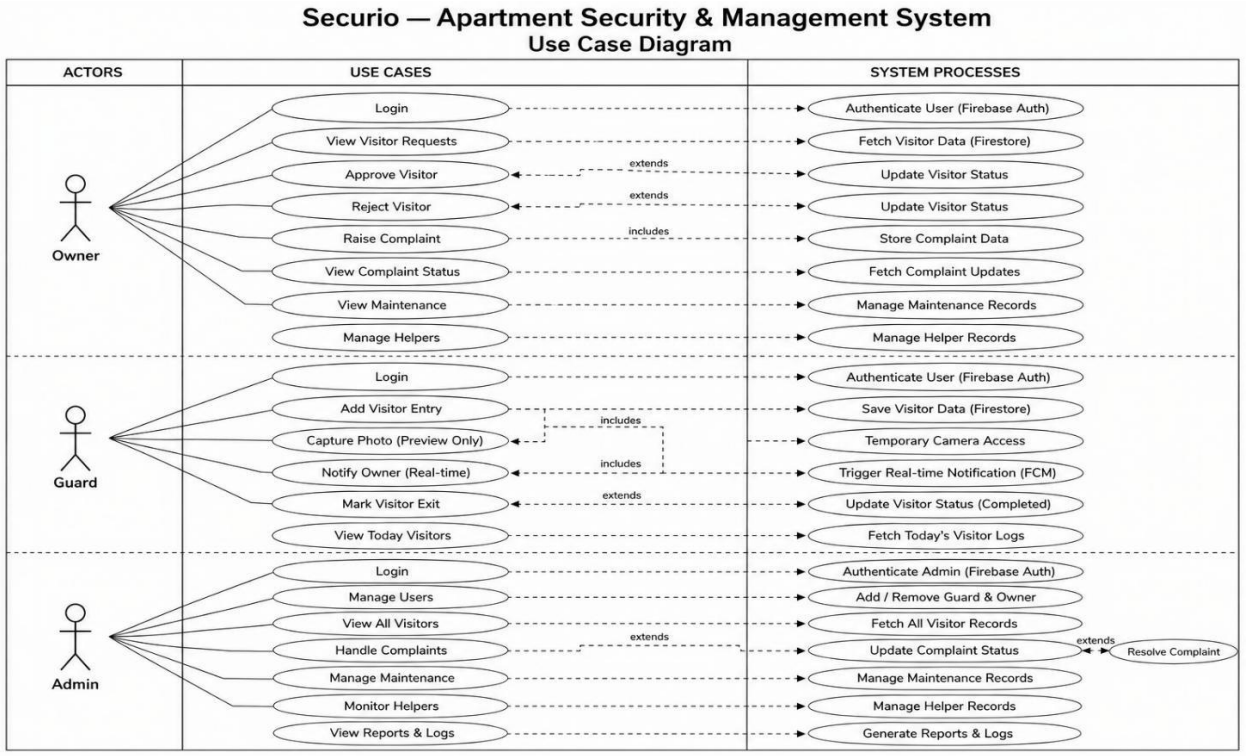


Figure 5.2 Data use

Securio – Use Case Diagram Description (Points)

- The system consists of three main actors: Admin, Guard, and Owner.
- The Owner can log in, view visitor requests, approve or reject visitors, raise complaints, check complaint status, view maintenance details, and manage house helpers.
- The Guard performs operational tasks such as logging in, adding visitor entries, capturing visitor photos (preview only), notifying the owner in real time, marking visitor exits, and viewing daily visitor records.
- The Admin manages the system by logging in, managing users (guards and owners), viewing all visitor records, handling and resolving complaints, managing maintenance data, monitoring helper records, and generating reports and logs.
- The system uses backend processes like Firebase Authentication for login, Firestore Database for storing and retrieving data, and Firebase Cloud Messaging (FCM) for real-time notifications.
- Visitor and complaint statuses are updated dynamically, ensuring real-time system functionality.
- Maintenance and helper records are efficiently managed within the system.
- The diagram includes “include” relationships for mandatory actions (e.g., adding a visitor includes capturing a photo and notifying the owner).

Figure 5.3 Data flow

Securio App – System Architecture

- Provides controlled access where each user has specific responsibilities and permissions.
- Ensures safe and reliable user authentication with role-based access control.
- Guards add visitor details and owners approve or reject requests in real time.
- Cloud-based architecture ensures high performance, reliability, and real-time updates.
- Sends instant alerts to users for visitor updates, approvals, and system activities.
- Users can raise complaints, and admins track and update their status efficiently.
- Admins assign maintenance dues, and owners can view and monitor payment status.
- Owners manage helper details, and guards verify entry based on permissions.
- Provides centralized control to monitor users, visitors, complaints, and reports.
- Stores and synchronizes all data in real time for fast and efficient access.

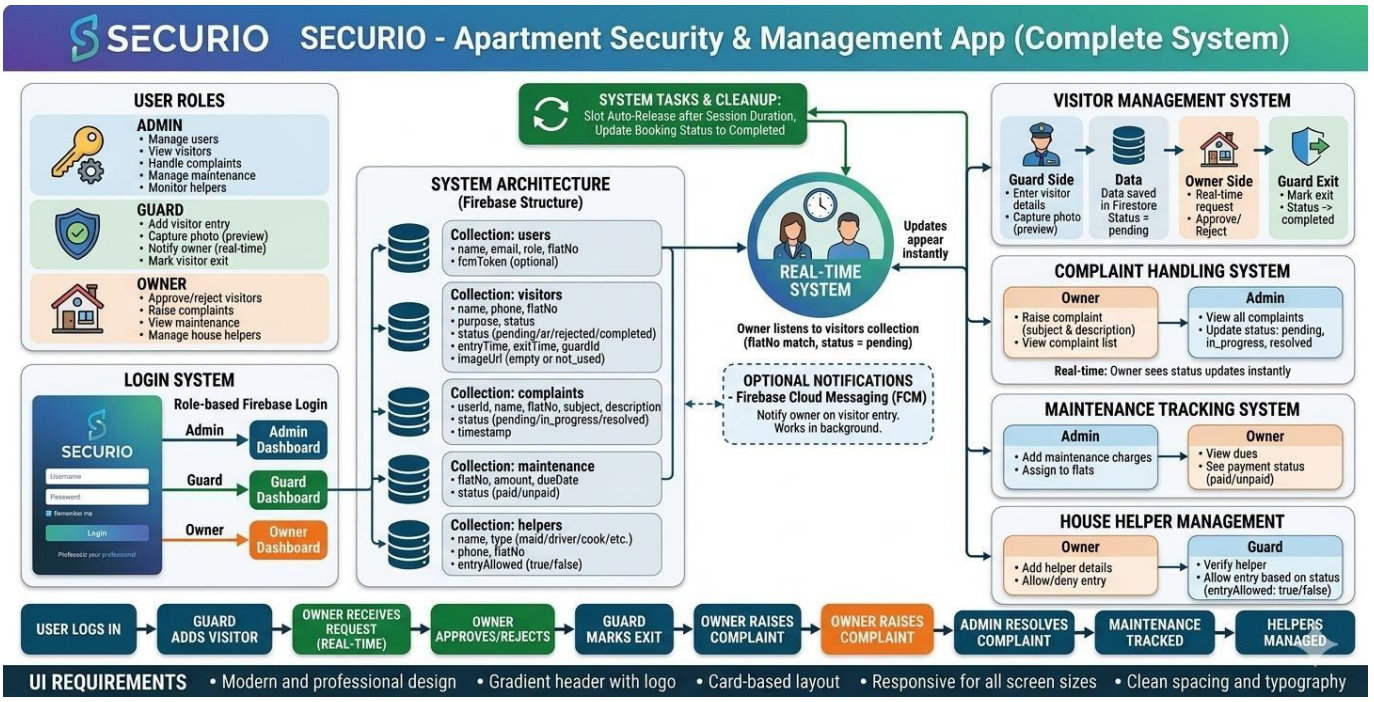


Figure 5.4 System Architecture

Core User Roles

- Admin: The high-level manager who handles user accounts (Guards/Owners), oversees all visitor logs, manages maintenance records, and resolves complaints.
- Guard: The frontline user responsible for logging visitor data, capturing entry photos (preview only), notifying owners, and marking final exits.
- Owner: The resident who receives real-time arrival alerts to approve or reject entry , raises service complaints, tracks maintenance payments, and manages their personal house helpers.

System Architecture (Firebase)

The system uses Cloud Firestore with a collection-based structure to ensure data is organized and synced:

- Users: Stores profiles, roles (Admin/Guard/Owner), and FCM tokens for notifications.
- Visitors: Tracks names, purpose of visit, status (pending/approved/completed), and entry/ exit timestamps.
- Complaints: Links descriptions and subjects to specific users with status tracking (Pending $\rightarrow$ Resolved).

- Maintenance: Manages financial data per flat, including due dates and payment status.
- Helpers: A database of service staff (maids, drivers) with specific "Entry Allowed" flags.

Key Management Modules

1. Guard enters data.
2. Firestore status becomes "Pending."
3. Owner receives a real-time request.
4. Upon approval, the Guard marks the exit to close the loop.

- Real-Time System: Utilizing a listener on the "Visitors" collection (filtered by flatNo), the app ensures owners see requests instantly without refreshing the page.
- Optional Notifications: Integrated via Firebase Cloud Messaging (FCM) to provide background alerts even when the app is closed.
- Maintenance & Complaints: A two-way street where Admins assign charges or update complaint progress, while Owners have immediate visibility into their specific records.

UI & UX Design Standards

- Visual Style: Modern, card-based layout featuring a gradient header and professional typography.
- Accessibility: Fully responsive design meant to scale across all Android screen sizes.
- Security: Role-based access control (RBAC) ensures users only see the dashboard relevant to their permissions.

Chapter 6: Project Scheduling

PROJECT TITLE: Securio

PROJECT GUIDE: Prof.Veena Gawde

INSTUTUTE & DEPARTMENT AP SHAH INSTITUTE OF TECHNOLOGY(CSE-Data Science)

DATE: 4-1-26

WBS NUMBER	TASK TITLE	TASK OWNER	START DATE	DUE DATE	DURATION(Weeks)	PERCENTAGE OF TASK COMPLETE
1	Project Conception and Initiation					
1.1	Group formation and Topic finalization. Identifying the scope and objectives of the Mini Project	Srushti Balsaraf,Shravani Bhilare	1-13-26	1-16-26	1	100%
1.2	Identifying the functionalities of the Mini Project	Sakshi Kale	1-16-26	1-20-26	1	100%
1.3	Discussing the project topic with the help of paper prototype.	Pratiksha ,Shravani ,Sakshi ,Srushti	1-20-26	2-3-26	2	100%
1.4	Designing the Graphical User Interface(GUI)	Srushti Balsaraf	2-3-26	2-17-26	2	100%
1.5	Presentation I	Pratiksha,Shravani,Sakshi,Srushti	2-17-26	2-24-26	1	100%
2	Project Design and Implementation					
2.1	Database Design	Pratiksha Jadhav	2-24-26	3-10-26	2	100%
2.2	Database Connectivity of all modules	Pratiksha Jadhav	3-10-26	3-17-26	1	100%
2.3	Integration of all modules and Report Writing	Shravani Bhilare	3-17-26	3-24-26	1	100%
2.4	Presentation II	Pratiksha,Shravani,Sakshi,Srushti	3-24-26	4-1-26	1	100%

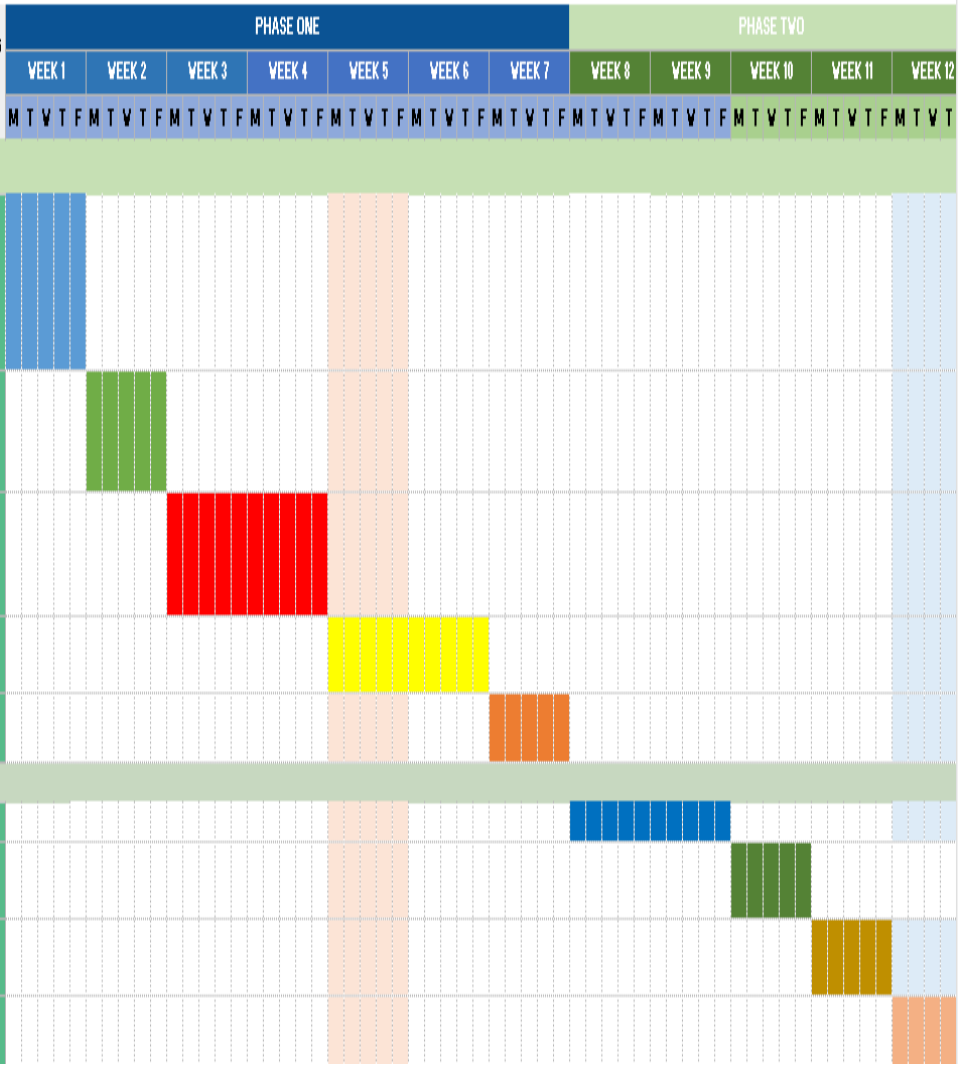


Figure 6.1: Gantt Chart

Chapter 7: Result

The **Securio – Apartment Security & Management App** was successfully developed and tested, demonstrating reliable and efficient performance across all its modules. The application includes a secure role-based login system that provides separate dashboards for Admin, Guard, and Owner, ensuring proper access control and usability.

The visitor management system operates effectively, allowing guards to add visitor details and enabling owners to approve or reject requests in real time. The complaint management module allows users to raise complaints and track their status, while the maintenance module provides accurate information about dues and payment status, ensuring transparency.

Additionally, Cloudinary integration enables efficient image handling, and the application offers a clean and responsive user interface for smooth navigation. Overall, the system performs efficiently with real-time updates, improved communication, and a user-friendly experience.

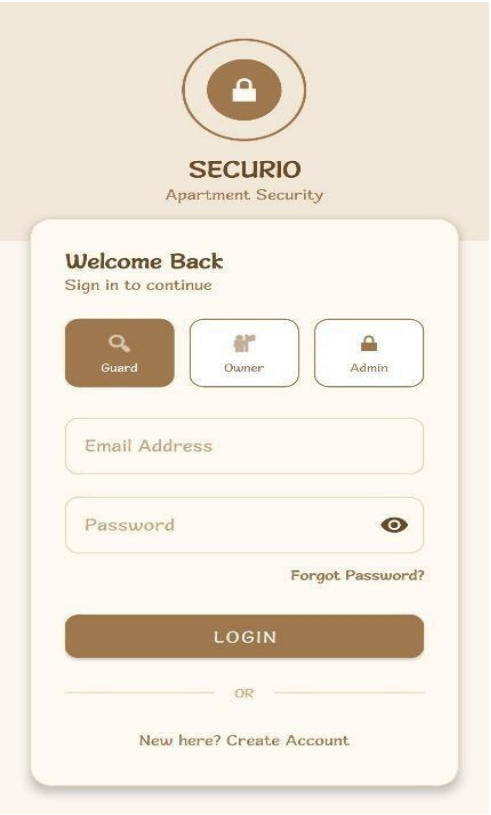


Figure 7.1: Sign-in page

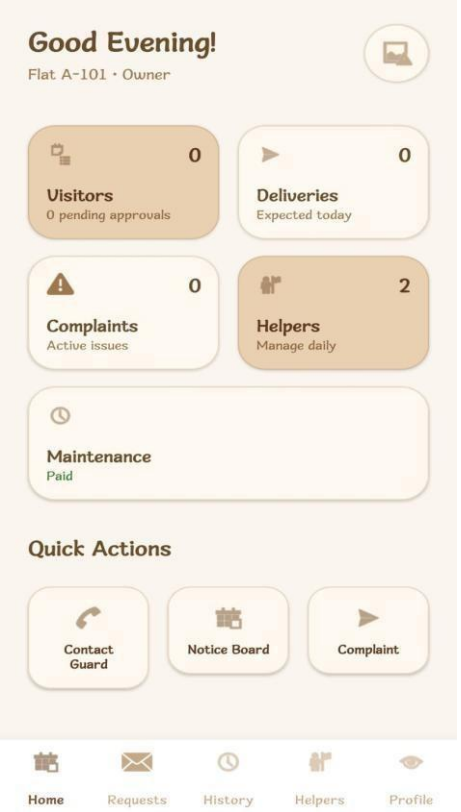


Figure 7.2: Owner dashboard

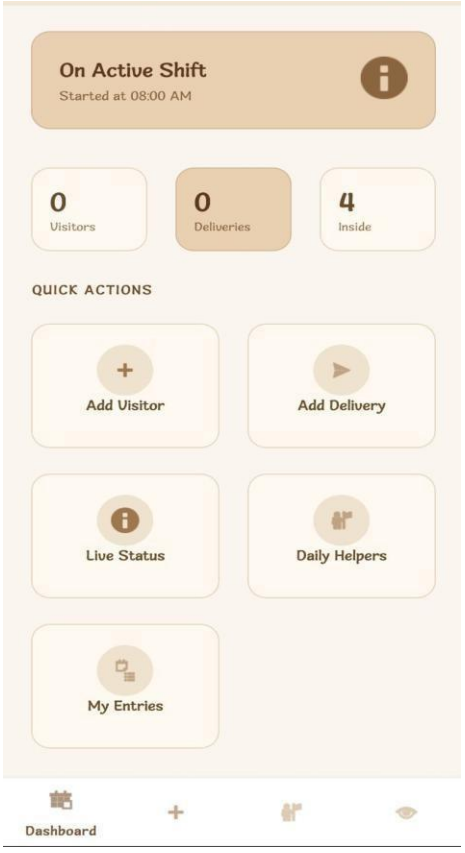


Figure 7.3: Guard dashboard

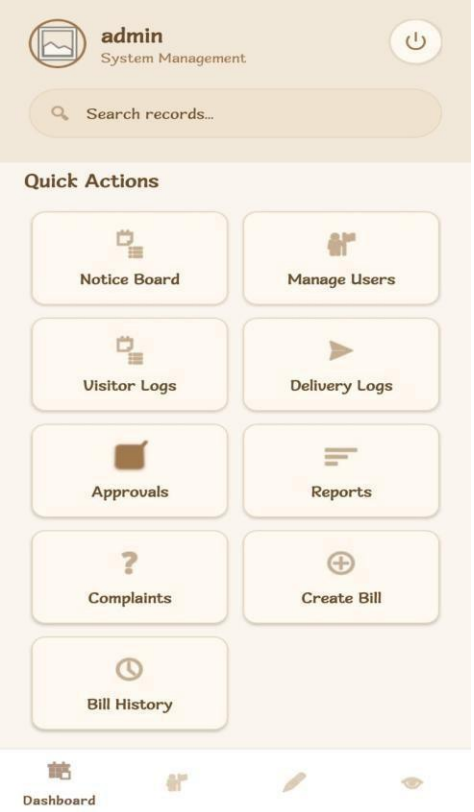
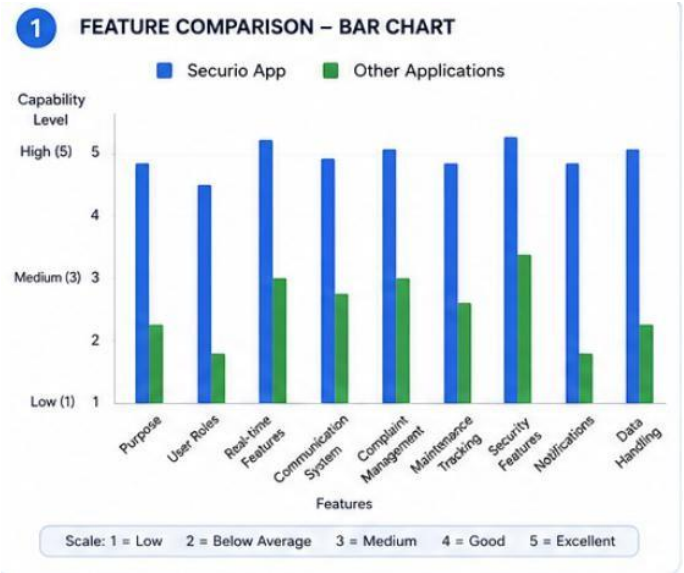
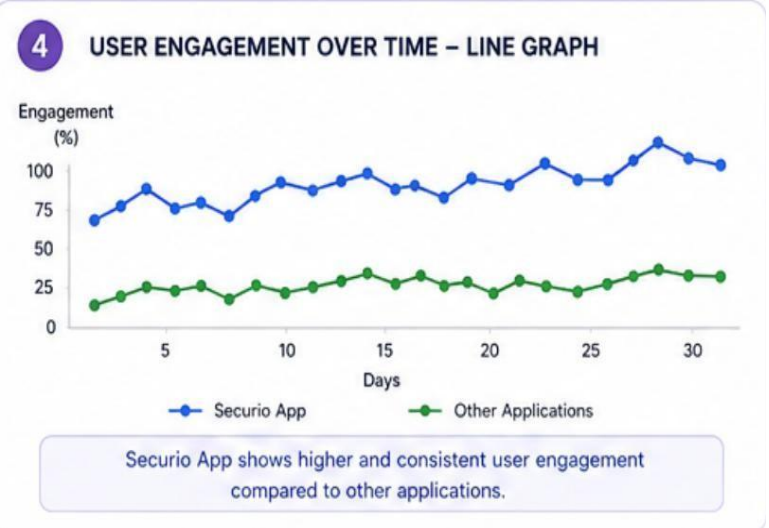


Figure 7.4: Admin dashboard





The diagram compares the Securio App with other utility-based applications using multiple visual charts to highlight differences in features, performance, and user engagement. It shows that Securio provides a comprehensive apartment security and management system with features like real-time visitor approval, role-based access (Admin, Guard, Owner), complaint handling, maintenance tracking, and helper management. In contrast, other apps are limited to specific utility functions such as location-based services or basic notifications. The graphs bar indicate that Securio has higher capability levels, better feature coverage (around 89%), and stronger overall performance compared to other apps (around 44%). Additionally, user engagement and daily active users are consistently higher in Securio, demonstrating better usability and adoption. Overall, the diagram proves that Securio is a more complete, secure, and interactive system, whereas other applications offer only partial functionality.

Chapter 8: Conclusion

The **Securio – Apartment Security & Management App** successfully addresses the major challenges faced by residential societies by providing a centralized and digital solution. By integrating key features such as visitor management, complaint handling, maintenance tracking, and house helper management into a single platform, the application significantly enhances security, improves communication, and simplifies overall society operations.

The use of modern technologies such as Firebase ensures real-time data synchronization, secure authentication, and efficient backend management. Additionally, Cloudinary integration enhances image handling by providing optimized storage and quick access. The role-based access system further improves usability by offering customized functionalities for Admin, Guard, and Owner, ensuring smooth coordination among all users.

The system is scalable, reliable, and user-friendly, making it highly suitable for modern residential societies. It reduces dependency on manual processes, increases transparency, and provides a seamless user experience. Overall, the application proves to be an effective solution for improving security and management in apartment complexes.

References

Firebase Documentation (Backend & Real-Time Database)

https://firebase.google.com/docs

Android Developer Documentation (App Development)

https://developer.android.com/docs

Cloudinary Documentation (Image Storage & Optimization)

https://cloudinary.com/documentation

YouTube (Learning Resources & Tutorials)

https://www.youtube.com/

GitHub (Code Reference & Version Control)

https://github.com/

Claude AI (Idea Support & Assistance)

https://claude.ai/